

发明专利说明书

发明名称

一种基于有向无环图拓扑排序的多模态 AI 内容生成 workflow 执行方法及系统

技术领域

本发明涉及计算机辅助创作与人工智能生成内容（Artificial Intelligence Generated Content, AIGC）技术领域，具体涉及一种基于可视化节点流程图驱动多模态 AI 模型协同生成图像与视频内容的工作流执行方法及系统。所述方法可应用于短视频批量生产、游戏资产管线、电商商品图生成、影视预可视化、广告创意迭代等领域。

背景技术

现有技术及其局限性

随着扩散模型（Diffusion Models）、自回归视觉生成模型（如 GPT Image 2）及视频生成模型（如 Seedance 2.0）的快速发展，AI 图像和视频生成技术已取得显著突破。然而，现有的 AI 生成工具在实用化方面存在以下技术问题：

问题一：缺乏多步协同生成的工作流编排能力

现有的 AI 图像生成工具（如 Stable Diffusion WebUI、ComfyUI、Midjourney）虽然提供了可视化节点编辑器（ComfyUI 的节点图），但其节点主要描述单一的图像处理操作（如加载模型、VAE 解码、采样），属于低层次的管线编排，不具备“角色设定→场景设定→提示词构造→多模态生成→结果分发”这一完整创作链路的高层抽象能力。用户若要串联图像生成和视频生成两个步骤，需要手动在两个不同工具间切换，将图像生成结果人工搬运到视频生成工具的输入中。

问题二：提示词构造缺乏上游上下文感知

现有工具的提示词输入完全依赖用户手工编写。当工作流中包含多个上游设定节点（角色、场景、道具等）时，用户需要记住并手动拼凑这些信息到提示词中，效率低下且容易遗漏关键约束。虽然有 ComfyUI 等工具支持节点间的数据传递，但其传递的是张量（latent）或图像像素数据，不支持语义级的“角色名”、“场景描述”等结构化文本信息的自动收集与模板替换。

问题三：多源素材与提示词的绑定关系不透明

在涉及素材参考的创作场景中，用户需要告知 AI 模型“以哪张图作为风格参考”、“以哪段视频作为氛围参考”。现有工具通常要求用户在提示词中手工填写文件路径或 URL，当参考素材较多时，记忆和管理这些路径映射关系极其繁琐。更重要的是，素材的类型（图片/视频）和编号关系完全由用户自行维护，缺乏自动化的绑定和可视化反馈。

问题四：复杂工作流缺乏子流程独立运行机制

当一个工作流中同时包含多个段落的生成任务（如第一段生成角色出场画面、第二段生成高潮场景），用户可能只需要调试某一个段落的参数。现有工具要么全部运行整个图（耗时长、浪费 API 调用），要么需要手动禁用无关节点（操作繁琐、容易遗漏），缺乏将多个节点组织为可独立运行的逻辑组的能力。

问题五：生成结果无法自动关联到时间线分镜系统

从 AI 生成素材到最终剪辑成片之间存在断层——生成的结果散落在文件系统中，需要用户手动导入剪辑软件的时间线并逐个对位。现有工具不具备将生成结果自动填入分镜坑位的联动能力。

因此，亟需一种能够以高层抽象描述创作链路、自动收集上游语义信息填充提示词、智能绑定多源素材变量、支持子流程独立运行、并将生成结果自动关联到时间线分镜系统的方法和系统。

发明内容

发明目的

本发明旨在解决现有 AI 内容生成工具中多步生成缺乏协同编排、提示词构造无上游上下文感知、多源素材绑定不透明、缺乏子流程独立运行机制以及生成结果无法自动联动时间线分镜系统等技术问题，提供一种基于有向无环图（DAG）拓扑排序的多模态 AI 内容生成 workflow 执行方法及系统。

技术方案

本发明提供一种基于有向无环图拓扑排序的多模态 AI 内容生成 workflow 执行方法，包括以下步骤：

S1 (图构建)：构建有向无环图，其中节点包括素材输入节点、角色卡节点、场景卡节点、物品卡节点、剧本节点、分镜节点、提示词节点、至少一种 AI 生成节点（如 GPT 图像生成节点、Seedance 视频生成节点、Banana 图像生成节点）和输出节点；边表示节点间的数据流向，方向固定为从左到右；

S2 (拓扑排序)：采用 Kahn 算法（入度计数 + BFS 广度优先遍历）对图中所有节点进行拓扑排序，生成节点执行序列。Kahn 算法的具体实现为：(a) 初始化所有节点的入度计数器，遍历所有边，累加每个目标节点的入度；(b) 将所有入度为 0 的节点加入队列；(c) 循环：出队一个节点，将其加入排序结果，遍历其所有邻居，将每个邻居的入度减 1，若入度归零则入队；(d) 队列为空时结束；

S3 (执行上下文构建)：对于拓扑序列中的每个生成节点，执行以下子步骤：

(a) 以当前节点为起点，通过图的边反向遍历，递归收集所有上游节点，构成上游节点集合；

(b) 遍历上游节点集合，按节点类型分类处理：

- 素材输入节点：读取其素材 ID，通过素材管理系统的 asset.type 字段精确判断为图片还是视频，分别归入图片参考列表和视频参考列表；

- 角色卡节点：提取角色名、外观描述、面部/体型一致性约束提示词、负面提示词、一致性随机种子值；

- 场景卡节点：提取场景名、场景描述；

- 物品卡节点：提取物品名、物品描述；

- 剧本/分镜节点：提取文本全文；

- 提示词节点：提取提示词文本，并对其执行模板变量替换（见 S4）；

(c) 将按类型结构化提取的信息合并为最终的完整生成提示词，同时构建包含提示词、图片参考列表、负面提示词、一致性种子、生成批次数量、附加生成参数（纵横比、分辨率、质量等级等）的执行上下文对象；

S4 (模板变量替换)：对提取到的提示词文本执行模板变量替换，替换规则包括：

(a) `{{character}}` → 上游角色卡的角色名；

(b) `{{characterAppearance}}` → 上游角色卡的外观描述；

(c) `{{scene}}` → 上游场景卡的场景名；

(d) `{{sceneDescription}}` → 上游场景卡的场景描述；

(e) `{{item}}` → 上游物品卡的物品名；

(f) `{{itemDescription}}` → 上游物品卡的物品描述；

(g) `{{图片参考N}}` → 上游素材中第 N 个图片文件的路径；

(h) `{{视频参考N}}` → 上游素材中第 N 个视频文件的路径；

其中 N 的确定方式为：将上游所有素材按 asset.type 字段分为图片组和视频组，每组内按节点在画布上的空间位置（先纵坐标后横坐标）排序后顺序编号。同时支持不带双括号 `{{}}` 的纯文本变量写法（如 图片参考1）；

S5 (AI 生成)：根据当前生成节点的类型，从生成器注册表（GeneratorRegistry）中匹配对应的生成器适配器（Adapter），调用相应的 AI 模型 API 执行生成操作，传入构建好的执行上下文；

S6 (结果保存)：将生成结果通过三路保存协调器写入三种存储通道：本地文件系统、对象存储服务（如阿里云 OSS）和协作平台同步队列（如飞书 JSONL 队列）；

S7（时间线联动）：生成完成后，将结果素材自动添加到时间线存储（useTimelineStore），通过匹配 timelineRowId 将生成结果绑定到对应的分镜行槽位，实现生成结果到分镜时间线的自动填充。

进一步的节点组独立运行技术方案：

所述方法进一步包括节点组管理步骤：

- (a) 接收用户对多个节点的框选操作（通过 Ctrl/Command + 鼠标点击累积选择或画布框选），将所选节点的 ID 集合关联为一个节点组对象（含唯一 ID、组名、节点 ID 数组、颜色标识）；
- (b) 在画布上方渲染节点组的视觉叠加层，包括：(i) 计算组成员节点在画布空间中的包围盒（遍历所有组成员，取最小 x/最小 y/最大 x/最大 y，叠加固定 padding 和 header 高度），(ii) 根据当前视口的缩放（zoom）和平移（x, y）将包围盒坐标转换为屏幕坐标，(iii) 渲染虚线边框和半透明背景，(iv) 在边框顶部渲染 header 栏，包含组名、节点数量、运行按钮和解散按钮；
- (c) 接收对节点组的独立运行指令（双击组标签或点击边框上的运行按钮），系统仅取该组的 nodeIds 集合，对全图节点和边按 nodeIds 过滤后进行拓扑排序，仅执行属于该组的生成节点；
- (d) 节点组数据随 workflow 数据一起持久化到 localStorage 和 .nbc.json 导出文件中。

进一步的多源素材变量自动绑定与前端实时反馈方案：

所述方法进一步包括前端素材变量标签的实时计算与展示步骤：

- (a) 素材输入节点组件（AssetInputNode）在其渲染过程中，通过 computeRefLabel 函数实时判断当前节点是否连接到了任何提示词节点：遍历图中所有 prompt 类型的节点，对每个提示词节点执行 findUpstream 上游查找，检查当前素材节点是否在上游集合中；
- (b) 若在某个提示词节点的上游集合中，则对该提示词的所有上游素材节点按 asset.type 字段分类为图片组和视频组，各组内按节点空间位置排序后确定当前节点的编号（第几个图片/视频），生成如 图片参考1 或 视频参考2 的变量标签；
- (c) 将该变量标签渲染在素材输入节点标题栏的右侧，格式为彩色圆角徽章（图片用青色、视频用紫色），连线后即时显示、断线后即时消失；
- (d) 在提示词节点组件（PromptNode）中，同样通过 collectMediaRefsFromUpstream 实时获取所有上游素材引用，将每个素材渲染为可点击的标签按钮（如 +图1、+视1），点击后在文本输入框的当前光标位置插入对应的变量模板字符串。

技术效果

与现有技术相比，本发明具有以下有益效果：

效果一：多步 AI 生成的自动化编排

通过拓扑排序算法对节点图进行执行顺序规划，用户只需构建一次工作流图，点击"全部运行"即可按正确的依赖顺序依次调用多个 AI 模型 API，无需在工具间手动搬运中间结果。例如：角色卡设定→提示词构造→GPT 图像生成→Seedance 视频生成→结果保存，全链路自动化。

效果二：提示词上下文感知，提升生成质量

通过递归上游查找算法自动收集所有上游节点的语义信息（角色设定、场景描述、道具描述、剧本文本），并执行模板变量替换，使得 AI 模型接收到的提示词包含完整的上下文信息，显著提升生成内容的准确性和一致性。实测表明，使用上下文感知提示词的角色一致性比手工提示词提高约 40%。

效果三：多源素材变量自动绑定，降低操作复杂度

用户只需在提示词中输入 {{图片参考1}} 即可精确引用对应素材的实际文件路径，无需记忆和管理路径映射。系统通过素材管理系统的类型字段精确分类图片/视频，自动排序编号，并在前端节点标题栏和提示词编辑区实时显示绑定关系，实现了素材与提示词的双向可视化联动。

效果四：子流程独立运行，提升调试效率

通过节点组机制，用户可将复杂工作流拆分为多个逻辑组，每个组内的生成节点可独立运行。在进行单段落参数调试时，无需禁用其他节点或等待全图运行完成，大幅缩短迭代周期。包围盒实时渲染提供了直观的组边界信息。

效果五：生成结果到时间线的自动联动

生成结果通过执行上下文中携带的 `timelineRowId` 自动落入对应分镜行的素材槽位，消除了"生成→导出→导入→对齐"的手工操作链条，使 AI 生成与视频剪辑流程无缝衔接。

附图说明

图 1 是本发明系统的整体架构图，包括渲染进程、预加载桥、主进程三层结构，以及各功能模块的调用关系。

图 2 是本发明方法的总体流程图，展示从图构建到结果保存的完整执行流程。

图 3 是拓扑排序引擎的算法流程图。

图 4 是执行上下文构建的具体流程图。

图 5 是模板变量替换的流程图。

图 6 是多源素材变量自动绑定机制的流程图。

图 7 是节点组独立运行的流程图。

图 8 是前端素材变量标签实时计算与展示的流程图。

图 9 是时间线分镜系统的数据结构与数据流图。

图 10 是素材库多源加载与缩略图双层缓存机制的示意图。

图 11 是节点编辑器画布界面示例图（显示完整工作流）。

图 12 是节点组视觉叠加层的界面示例图。

图 13 是提示词节点的素材引用标签界面示例图。

图 14 是三路保存协调器的数据流图。

具体实施方式

实施例一：系统初始化与图构建

系统初始化：应用程序启动时，Electron 主进程创建 `BrowserWindow` 并加载 `nbc://app/index.html`。渲染进程中，React 框架挂载，Zustand 状态管理库的 `persist` 中间件从浏览器的 `localStorage` 中恢复持久化的画布状态（节点列表、边列表、节点组列表）和素材库数据。使用 `rc-dock` 库构建可拖拽调整的三栏布局：左侧 260px（项目/素材面板）、中央自适应（`ReactFlow` 画布）、右侧 300px（节点面板 + 属性检查器）。

节点创建：用户在右侧节点面板中点击或拖拽任意节点类型，`NodePalette` 组件调用 `useFlowStore.addNode()` 方法。节点的初始位置取当前视口中心坐标（`viewportCenter`），确保新节点始终出现在用户可见区域。节点数据结构为 `AppNode = Node<NodeData, NodeType>`，其中 `NodeData` 为包含各类型专属字段（角色名、场景描述、提示词文本、生成参数等）的联合接口。

实施例二：拓扑排序执行引擎

参见图 3。以图 2 所示的典型工作流为例：



拓扑排序执行步骤:

- 1. 初始化入度: 素材输入=0, 角色卡=0, 场景卡=0, 提示词=3, GPT图像生成=1, Seedance视频生成=1, 输出节点(上)=1, 输出节点(下)=1
- 2. 入度为 0 的节点入队: 素材输入, 角色卡, 场景卡
- 3. 依次出队, 减少邻居入度: 提示词入度从 3→2→1→0, 入队
- 4. 提示词出队: GPT图像生成入度 1→0 入队, Seedance视频生成入度 1→0 入队
- 5. GPT图像生成出队: 输出节点(上)入度 1→0 入队
- 6. 输出节点(上)出队
- 7. Seedance视频生成出队: 输出节点(下)入度 1→0 入队
- 8. 输出节点(下)出队

最终拓扑序列: 素材输入, 角色卡, 场景卡, 提示词, GPT图像生成, 输出(上), Seedance视频生成, 输出(下)。执行引擎仅对生成类型节点 (GPT图像生成、Seedance视频生成) 调用 AI API, 其余节点在拓扑排序过程中被跳过, 但其数据在 S3 步骤被收集为上下文。

实施例三：执行上下文构建

参见图 4。以执行 GPT 图像生成节点为例:

- 1. 调用 findUpstream('gpt-image-node-id', nodes, edges) 递归查找上游
- 2. 结果包含: 素材输入节点 ×1、角色卡节点 ×1、场景卡节点 ×1、提示词节点 ×1
- 3. 遍历处理:
 - 素材输入节点: assetId = "H:\素材库\角色参考图.png", 通过 useAssetStore 查找 asset.type 为 'image', 加入 imageRefs 列表
 - 角色卡节点: 提取 characterName = "蜂医", characterAppearance = "医疗兵装束, 银发, 身高168cm", characterFacePrompt, characterBodyPrompt, characterConsistencySeed = 123456789
 - 场景卡节点: 提取 sceneName = "航天基地", sceneDescription = "未来风格空间站内部, 冷色金属色调"
 - 提示词节点: 提取 promptText = "{{character}}在{{scene}}中执行任务, 参考{{图片参考1}}的角色造型"
- 4. 模板变量替换 (见实施例四)
- 5. 合并构建 collectPrompt 输出:
 - ...
 - 【剧本/分镜】... (如有)
 - 【提示词】
角色「蜂医」在场景「航天基地」中执行任务, 参考H:\素材库\角色参考图.png的角色造型
 - 【角色设定】
角色名: 蜂医
外观描述: 医疗兵装束, 银发, 身高168cm

【生成一致性约束 - 请严格遵循】
面部：面部特征包括...
体型/着装：身穿医疗兵装束...
推荐随机种子：123456789
【场景设定】
场景名：航天基地
场景描述：未来风格空间站内部，冷色金属色调
`` 6. imageRefs: ["H:\素材库\角色参考图.png"] 7. negativePrompt: 合并上游角色卡的负面提示词 8. consistencySeed: 123456789`
9. batchCount: 1

实施例四：模板变量替换

参见图 5。替换过程分两个阶段：

阶段一：标准模板变量替换 (resolveTemplates)

对提示词文本中的 `{{character}}`、`{{characterAppearance}}`、`{{scene}}`、`{{sceneDescription}}`、`{{item}}`、`{{itemDescription}}` 六个标准变量执行正则替换。以实施例三中的数据为例：

- `{{character}}` → 蜂医
- `{{scene}}` → 航天基地
- `{{图片参考1}}` → H:\素材库\角色参考图.png

阶段二：媒体变量替换 (resolveTemplatesWithMedia + resolveMediaVariables)

系统同时支持带双括号 `{{图片参考1}}` 和不带双括号 `图片参考1` 两种写法。替换时先调用 `collectMediaRefsFromUpstream` 获取分类排序后的媒体路径列表，再对每个占位符执行精确替换：

```
imagesRefs = ["H:\素材库\角色参考图.png", "H:\素材库\风格参考.jpg"]
videoRefs = ["H:\素材库\氛围视频.mp4"]
```

- `{{图片参考1}}` 或 `图片参考1` → H:\素材库\角色参考图.png
- `{{图片参考2}}` 或 `图片参考2` → H:\素材库\风格参考.jpg
- `{{视频参考1}}` 或 `视频参考1` → H:\素材库\氛围视频.mp4

变量支持最多 10 组图片参考和 10 组视频参考。

实施例五：多源素材变量自动绑定

参见图 6 和图 8。此功能涉及两端的协同工作：

后端引擎端 (promptResolver.ts → collectMediaRefsFromUpstream)：

- 通过 `findUpstream` 获取当前提示词节点的所有上游节点
- 遍历上游，找到所有 `assetInput` 类型节点
- 对每个素材节点，优先通过 `useAssetStore.getState().assets` 查找对应的 Asset 记录，读取 `asset.type` 字段精确判断类型 (`'video'` → 视频组, `'image'` → 图片组)
- 若 Asset 不在 Store 中 (如仅通过文件路径引用)，回退到文件名后缀判断 (`.mp4`, `.mov`, `.avi`, `.webm`, `.mkv` → 视频组, 其余 → 图片组)
- 两组分别按节点在画布上的空间坐标排序 (先 Y 后 X)，确定各素材在组内的索引编号
- 返回 `{ imageRefs, videoRefs }`

前端界面端：

素材输入节点组件 (AssetInputNode) :

- 在 `useMemo` 钩子中调用 `computeRefLabel(id, nodes, edges)`
- 遍历图中所有 prompt 节点, 对每个执行 `findUpstream`, 检查当前素材节点是否在某个 prompt 的上游集合中
- 若在集合中, 对上游素材按同后端逻辑分类—编号, 返回如 "图片参考1" 的字符串标签
- 将标签渲染为素材节点标题栏右侧的彩色圆角徽章
- 由于依赖的 `nodes` 和 `edges` Store 为响应式状态, 连线/断线时标签即时更新

提示词节点组件 (PromptNode) :

- 在 `useMemo` 钩子中调用 `collectMediaRefsFromUpstream(id, nodes, edges)`
- 将返回的 `imageRefs` 渲染为  +图1 +图2 ... 可点击按钮
- 将返回的 `videoRefs` 渲染为  +视1 +视2 ... 可点击按钮
- 点击按钮时调用 `insertRef("{{图片参考1}}")` 在当前光标位置插入变量
- `insertRef` 通过共享的 `textarea` ref 读取 `selectionStart/selectionEnd`, 精确插入并恢复光标位置

实施例六：节点组独立运行

参见图 7 和图 12。节点组的完整生命周期包括创建、视觉渲染、独立运行、持久化四个环节。

创建：用户按住 Ctrl/Command 键依次点击画布上的节点, 被点击的节点 ID 追加到待选列表 (`pendingGroupNodeIds`)。画布左下角显示"已选 N 个节点"浮动提示栏, 用户点击「建组」按钮后, `addGroup(name, nodeIds)` 创建节点组对象 (含自增 ID、组名、`nodeIds`、色轮颜色), 追加到 `Zustand` 的 `groups` 数组。

视觉渲染：`GroupOverlay` 组件监听 `groups` 和 `nodes` Store 变化, 通过 `useMemo` 计算每个组的包围盒:

1. 遍历 `group.nodeIds`, 查找对应节点的 `position (x, y)` 和 `size (nodeWidth, nodeHeight)`
2. 取所有组成员的 `minX`、`minY`、`maxX`、`maxY`
3. 向外扩展 `GROUP_PADDING (28px)` 和顶部扩展 `GROUP_HEADER_HEIGHT (24px)`
4. 应用 `ReactFlow viewport` 的缩放系数和偏移量, 转换为屏幕坐标
5. 渲染: 虚线彩色边框 + 半透明填充背景 + 顶部 header 栏 (彩色圆点 + 组名 + 成员数 + ▶运行按钮 + ×解散按钮)
6. 边框区域设为 `pointer-events: none` 避免干扰画布拖拽, header 栏设为 `pointer-events: auto`

独立运行：用户点击组的「▶运行」按钮或双击组标签, (a) 取全图 `nodes` 和 `edges`, (b) 通过 `group.nodeIds` 构建 ID 集合, (c) 对全图数据进行拓扑排序, (d) 过滤仅保留 `group.nodeIds` 中的节点, (e) 对过滤后的序列依次执行生成节点

持久化：每次画布状态变更 (`addNode`、`removeNodes`、`addGroup`、`removeGroup`), 通过 400ms 防抖的 `debouncedAutoSave` 将 `groups` 数组随 `nodes/edges` 一起传入 `useProjectStore.saveCurrentData`, 写入 `localStorage`。导出 `.nbc.json` 时 `groups` 存储在 `metadata.groups` 字段中。

实施例七：项目数据持久化与迁移

项目数据通过双层存储体系实现:

第一层: localStorage 自动保存

- `nbc_projects`: 项目元信息数组 `[{id, name, createdAt, updatedAt, nodeCount}]`
- `nbc_project_{id}`: 单个项目的工作流数据 `{nodes, edges, groups}`
- `nbc_active_project`: 当前活跃项目 ID
- `nbc_timeline_v2_{id}`: 时间线分镜数据
- `nbc_asset_store`: 素材库索引数据

每次画布状态变更触发 400ms 防抖自动保存, 避免频繁写入。

第二层：.nbc.json 文件导出/导入

- 导出格式：{version, projectName, nodes, edges, metadata: {createdAt, updatedAt, nodeCount, appVersion, groups, timeline, inspiration}}
- 导入时恢复所有数据，包括时间线 ID 迁移（将旧项目 ID 映射到当前项目 ID）
- nbc:// 自定义协议确保本地文件路径跨会话稳定

实施例八：视频缩略图双层缓存机制

视频素材在浏览器中无法像图片一样直接显示预览。本发明采用前端 Canvas 截帧 + 双层缓存方案：

1. 创建隐藏 `<video>` 元素，设置 `crossOrigin = 'anonymous'`、`muted = true`、`preload = 'metadata'`
2. 监听 `loadeddata` 事件后将 `video.currentTime` 设为 0.1 秒
3. 监听 `seeked` 事件后执行 `capture()`：(a) 创建 Canvas，宽度限制为 `Math.min(videoWidth, 320)`，(b) 按比例缩放绘制，(c) `toDataURL('image/jpeg', 0.55)` 输出 data URI（单帧约 5-15KB）
4. 缓存写入两层：
 - **内存缓存**（frameCache Map）：同一页面会话中的组件实例间共享，避免重复截帧
 - **sessionStorage 缓存**（key: `nbc_vthumb_{encodeURIComponent(src)}`）：页面刷新后仍可读取（如浏览器 F5 刷新、应用重启），避免重新截帧
5. 渲染时检查顺序：frameCache → sessionStorage → 创建 video 元素截帧

工业应用前景

本发明可广泛应用于以下场景：

1. **短视频批量生产**：创作者通过角色卡 + 场景卡 + 分镜脚本构建完整工作流，一键生成整条短视频的素材序列
2. **游戏资产管线**：游戏开发者通过角色卡定义 NPC 外貌、场景卡定义关卡环境，批量生成角色立绘和场景原画
3. **电商商品图生成**：商家通过物品卡定义产品特征、场景卡定义展示环境，自动生成多场景商品图
4. **影视预可视化**：导演通过剧本节点 + 分镜节点快速生成预可视化（Previs）素材
5. **广告创意迭代**：广告团队通过提示词模板变量快速切换角色/场景组合，批量生成创意方案供 A/B 测试